

CHAPTER 14

14. NestJS 캐싱 + 스케줄링 — Redis 캐시와 @Cron 배치

Node.js · Express · NestJS 강의 · 2026

게시판에 인기글 목록 같은 화면이 있다고 하자. 이 목록은 모든 글을 조회수로 정렬해 상위 다섯 개를 뽑는다. 글이 수만 개면 이 정렬은 결코 가볍지 않은데, 정작 결과는 몇 초 사이에 거의 바뀌지 않는다. 매 요청마다 같은 무거운 쿼리를 다시 도는 건 분명한 낭비다. 한 번 계산한 결과를 잠깐 보관해 두고, 그동안 들어오는 요청에는 그 사본을 내주는 것 — 이것이 캐싱(caching)이다.

또 하나, 어떤 일은 요청과 무관하게 정해진 시각에 알아서 돌아야 한다. 매일 자정에 그날의 통계를 집계해 적재하는 배치, 한 시간마다 만료된 세션을 청소하는 작업 같은 것들이다. 누가 API를 부르지 않아도 정해진 시각에 맞춰 작업을 실행하는 것이 스케줄러(scheduler)다.

이 장에서는 NestJS 게시판에 이 둘을 붙인다. 캐시는 Redis에 두고, 스케줄은 @Cron 데코레이터로 건다. 12:13장에서 만든 Prisma-모듈 구조를 그대로 쓰되, 여기에 캐싱 계층과 배치 계층을 더하는 셈이다.

이 자료는 개념 예습용이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 무엇을 배우는가에 초점을 둔다.

이 장에서 배우는 것

- 캐시 어사이드(cache-aside) 패턴 — 읽기는 "캐시 먼저, 없으면 DB", 쓰기는 "DB 바꾼 뒤 캐시 삭제"
- 전역 CacheModule 에 Redis 스토어를 연결하고 CACHE_MANAGER 를 주입해 get / set / del 쓰기
- TTL(Time To Live) — 캐시에 유효기간을 뒤 오래된 데이터가 영원히 남지 않게 하는 법
- @nestjs/schedule 의 @Cron 으로 메서드를 주기 실행하고, 같은 로직을 수동 엔드포인트로도 부르는 법

내용은 두 갈래로 나뉜다. 한쪽은 게시판 읽기를 빠르게 만드는 캐싱, 다른 쪽은 정해진 시각에 스스로 도는 스케줄링이다. 두 기능을 켜는 스위치는 모두 루트 모듈 한곳에 모여 있다.

주제	핵심
두 기능 켜기	CacheModule (Redis) + ScheduleModule.forRoot() 등록
캐시 어사이드	캐시 먼저 읽고, 쓰면 무효화
캐시 무효화	글 생성·조회수 증가 시 관련 캐시 삭제
크론 스케줄	@Cron 으로 통계 스냅샷을 주기 적재
한 로직, 두 입구	집계 본체를 빼서 크론·수동 양쪽이 호출

핵심 개념

캐시 어사이드(cache-aside) — 옆에 따로 둔 사본

캐싱 방식에는 여러 가지가 있지만, 가장 흔하고 직관적인 것이 **캐시 어사이드**다. 이름 그대로 캐시를 본체(DB) "옆에(aside)" 따로 두고, 애플리케이션이 둘을 직접 오가며 관리한다. 규칙은 단 두 줄이다.

읽기: 캐시 확인 → 있으면 그대로 반환(hit) / 없으면 DB 조회 후 캐시에 저장(miss)
 쓰기: DB 변경 → 관련 캐시 삭제(del) ← "오래된(stale) 캐시"를 막는 핵심

읽기 쪽은 쉽다. 캐시에 있으면 DB를 건드리지 않고 바로 내주고(hit), 없으면 그때 DB에서 가져와 캐시에 넣어 둔다(miss). 그래서 **같은 요청이 두 번째부터는 빨라진다**.

까다로운 쪽은 쓰기다. DB의 글이 바뀌었는데 캐시에는 옛날 사본이 그대로 남아 있으면, 사용자는 갱신된 데이터를 한참 못 본다. 이걸 **stale(상한) 캐시**라 한다. 캐시 어사이드의 핵심은 쓰기가 일어나면 **관련 캐시를 지워(del)** 버리는 것이다. 지워 두면 다음 읽기가 자연스럽게 miss가 되고, DB에서 최신 값을 다시 채운다. 이 장의 `incrementView` (조회수 증가)와 `create` (글 생성)가 바로 이 무효화를 한다.

TTL — 캐시에도 유통기한이 있다

지운다고 해도, 외부 요인으로 데이터가 바뀌는 경우까지 일일이 무효화하긴 어렵다. 그래서 캐시에는 보통 **유효기간**을 함께 건다. 이게 **TTL(Time To Live)** 이다. 이 장은 TTL을 `30_000` 밀리초, 즉 **30초**로 뒀다. 한 번 채운 캐시는 30초가 지나면 자동으로 사라지고, 그 뒤 첫 요청은 다시 DB를 친다. 무효화가 안전망이라면, TTL은 "그래도 최대 30초면 최신화된 다"는 또 한 겹의 보장이다.

크론 스케줄 — 시계를 보고 도는 작업

`@Cron` 은 **크론식(cron expression)** 으로 실행 시각을 정한다. 크론식은 분·시·일·월·요일 다섯 칸으로 주기를 표현하는 오래된 유닉스 표기법이다. 예를 들어 `0 0 * * *` 은 "매일 0시 0분"이다. NestJS는 자주 쓰는 주기를

`CronExpression` 이라는 상수로 미리 묶어 두었다. 이 장은 데모용으로 `CronExpression.EVERY_MINUTE` (매분)를 쓴다. 실무라면 `EVERY_DAY_AT_MIDNIGHT` 나 `"0 0 * * *"` 로 매일 한 번 도는 게 보통이다.

콜아웃 — 한 로직, 두 입구

스케줄러로만 도는 작업은 디버깅이 번거롭다. 1분을 기다려야 한 번 도니까. 그래서 이 장은 집계 로직을 `capture()` 라는 **별도 메서드로 빼** 두고, 크론(`snapshot`)도 그걸 부르고 수동 엔드포인트(`POST /stats/run`)도 그걸 부른다. 핵심 로직은 한 곳에 있고 부르는 **입구만 둘**인 구조다. 이렇게 빼 두면 테스트와 즉시 실행이 쉬워진다.

준비

PostgreSQL과 Redis 두 가지가 모두 떠 있어야 한다. 캐시 스토어가 Redis이므로, **Redis가 꺼져 있으면 앱이 부팅조차 못 한다.**

```
# 빈 DB 생성
createdb nest_cache

# Redis 설치 & 실행 (macOS / Homebrew)
brew install redis
brew services start redis      # 백그라운드 상시 실행 (또는 포그라운드: redis-server)
```

그다음 의존성을 깔고 환경변수를 채운 뒤 마이그레이션을 돌린다.

```
npm install
cp .env.example .env
npx prisma migrate dev--name init
```

.env 는 세 가지를 본다. DB 접속 정보와 Redis 호스트·포트, 그리고 서버 포트다.

```
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/nest_cache?schema=public"
REDIS_HOST=localhost
REDIS_PORT=6379
PORT=3000
```

USER:PASSWORD 는 본인 로컬 PostgreSQL 계정으로 바꾼다. 준비가 끝나면 서버를 띄운다.

```
npm run start:dev      # → http://localhost:3000
```

콘솔에 1분마다 [cron] 스냅샷 저장 ... 로그가 찍히면 스케줄러가 도는 것이다.

왜 **dotenv/config** 를 맨 위에서 불러오나 — 앱 진입점의 첫 줄이 `import "dotenv/config"` 다. 루트 모듈이 `process.env.REDIS_HOST` 를 읽어 Redis에 붙는데, 모듈이 평가되기 **전에** `.env` 가 `process.env` 로 올라와 있어야 한다. 그래서 어떤 모듈보다도 먼저, 진입점 맨 윗줄에서 환경변수를 로드한다.

두 기능을 켜는 스위치 — 모듈 등록

캐싱과 스케줄링은 각각 모듈 하나를 `imports` 에 등록하는 것으로 켜진다. 루트 모듈을 먼저 보자.

```
@Module({
  imports: [
    // Redis 캐시 - 전역 등록. 어디서든 CACHE_MANAGER 주입으로 get/set/del.
    CacheModule.registerAsync({
      isGlobal: true,
      useFactory: async () => ({
        store: await redisStore({
          socket: {
            host: process.env.REDIS_HOST?? "localhost",
            port: Number(process.env.REDIS_PORT?? 6379),
          },
          ttl: 30_000, // 기본 TTL 30초(ms) - 개별 set에서 덮어쓸 수 있다
        }),
      }),
    ),
  ],
})
// @Cron 데코레이터를 동작시키려면 한 번 등록해야 한다.
ScheduleModule.forRoot(),
PrismaModule,
PostsModule,
StatsModule,
],
})
export class AppModule {}
```

캐시 등록부터 뜯어보자.

- **CacheModule.registerAsync** — 캐시 모듈을 비동기로 등록한다. Redis에 붙는 일(`redisStore(.)`)이 `await` 를 동반하는 비동기라서, 동기 `register` 가 아니라 `registerAsync` 에 팩토리 함수를 넘긴다.
- **isGlobal: true** — 한 번 등록하면 모든 모듈에서 별도 import 없이 캐시를 주입받을 수 있다. 캐시는 게시판·통계 어디서나 쓸 법한 횡단 관심사라, 전역으로 두는 게 편하다.
- **store: await redisStore({...})** — 캐시의 저장소를 Redis로 지정한다. `cache-manager-redis-yet` 이 제공하는 `redisStore` 가 `.env` 의 호스트·포트로 Redis에 연결한다. 이 한 줄이 "캐시를 어디에 둘 것인가"의 답이다. 이 store를 메모리로 바꾸면 코드 나머지는 그대로 둔 채 인메모리 캐시가 된다.
- **ttl: 30_000** — 기본 유효기간 30초(밀리초 단위). 개별 `set` 호출에서 다른 값을 주면 그 값이 우선한다.

스케줄 쪽은 더 단순하다.

- **ScheduleModule.forRoot()** — 이 한 줄이 `@Cron` 데코레이터를 깨운다. 이게 없으면 서비스에 `@Cron` 을 붙여도 아무 일도 일어나지 않는다. 이 한 줄을 등록해야 NestJS가 앱 시작 시 스케줄 메타데이터를 훑어 타이머를 건다.

`PrismaModule` 은 12장처럼 `@Global()` 로 선언돼 있어 DB 접근을 전역에 깔고, 게시판·통계 모듈이 각 도메인을 담는다.

cache-aside 캐싱 — 읽기와 무효화

캐싱의 모든 로직은 게시판 서비스 한곳에 모여 있다. 먼저 생성자와 상수를 보자.

```
@Injectable()
export class PostsService {
  private readonly POPULAR_KEY = "posts:popular";
  private readonly TTL = 30_000; // 30초 (ms)

  constructor(
    @Inject(CACHE_MANAGER) private readonly cache: Cache,
    private readonly prisma: PrismaService
  ) {}
```

- **@Inject(CACHE_MANAGER) private readonly cache: Cache** — 전역 `CacheModule` 이 제공하는 캐시 매니저를 주입받는다. `PrismaService` 는 클래스 이름으로 주입되지만, 캐시 매니저는 클래스가 아니라 `CACHE_MANAGER` 라는 토큰으로 등록돼 있어 `@Inject(.)` 로 명시해 꺼낸다. 이 `cache` 하나로 `get · set · del` 을 모두 한다.
- **POPULAR_KEY = "posts:popular"** — 인기글 캐시를 저장할 키. 캐시는 키-값 저장소라, 무엇을 어떤 이름으로 넣을지 키를 정해야 한다. 단건 글은 `post:${id}` 처럼 id를 붙여 글마다 다른 키를 쓴다. 콜론으로 네임스페이스를 나누는 건 Redis의 관례다.

읽기 — 인기글 (cache-aside의 교과서)

```
// 인기글 - "비싼 정렬" 결과를 캐싱(cache-aside). source로 cache/db를 확인할 수 있다.
async popular() {
  const cached = await this.cache.get(this.POPULAR_KEY);
  if (cached) return { source: "cache", items: cached };

  const items = await this.prisma.post.findMany({
    orderBy: { viewCount: "desc" },
    take: 5,
  });
  await this.cache.set(this.POPULAR_KEY, items, this.TTL);
  return { source: "db", items };
}
```

캐시 어사이드 읽기 패턴이 그대로 코드가 됐다.

1. **this.cache.get(this.POPULAR_KEY)** — 먼저 캐시를 본다. 값이 있으면(hit) DB를 건드리지 않고 `source: "cache"` 로 즉시 반환한다.
2. 없으면(miss) Prisma로 조회수 내림차순 상위 5개를 뽑는다. 이게 "비싼 쿼리"다.
3. **this.cache.set(this.POPULAR_KEY, items, this.TTL)** — 그 결과를 캐시에 넣는다. 세 번째 인자 `this.TTL (30초)`을 주면 30초 뒤 자동 만료된다. 그리고 `source: "db"` 로 반환한다.

`source` 필드는 학습용이다. 같은 요청을 두 번 보내 보면, 처음엔 `"db"`, 30초 안의 두 번째엔 `"cache"` 가 나온다. 캐시가 실제로 먹는지 눈으로 확인할 수 있다.

읽기 — 단건 조회 (글마다 다른 키)

```
async findOne(id: number) {
  const key = `post:${id}`;
  const cached = await this.cache.get(key);
  if (cached) return { source: "cache", post: cached };

  const post = await this.prisma.post.findUnique({ where: { id } });
  if (!post) throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
  await this.cache.set(key, post, this.TTL);
  return { source: "db", post };
}
```

흐름은 `popular` 과 똑같고, 키만 다르다. 글마다 캐시가 독립이어야 하니 `post:${id}` 로 id를 키에 박는다. 1번 글은 `post:1`, 2번 글은 `post:2` 에 따로 저장된다. 없는 글을 부르면 `NotFoundException` 을 던져 404로 응답한다. 이때는 캐시에 아무것도 넣지 않는다 — 없는 걸 캐시할 이유가 없다.

쓰기 — 생성과 무효화

```
async create(dto: CreatePostDto) {
  const post = await this.prisma.post.create({ data: dto });
  await this.cache.del(this.POPULAR_KEY); // 목록이 바뀌었으니 캐시 무효화
  return post;
}
```

글을 새로 만들면 인기글 목록이 달라질 수 있다. 그래서 DB에 넣은 직후 `this.cache.del(this.POPULAR_KEY)` 로 인기글 캐시를 지운다. 지워 두면 다음 `popular()` 호출이 miss가 되어 최신 목록을 다시 계산한다. 이게 stale 캐시를 막는 무효화다. (단건 키는 새 글이라 아직 캐시될 일이 없으니 건드리지 않는다.)

쓰기 — 조회수 증가와 이중 무효화

```
// 조회수 증가 - DB를 바꿨으니 관련 캐시를 무효화(stale 방지)
async incrementView(id: number) {
  const post = await this.prisma.post
    .update({ where: { id }, data: { viewCount: { increment: 1 } } })
    .catch(() => {
      throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
    });
  await Promise.all([
    this.cache.del(`post:${id}`),
    this.cache.del(this.POPULAR_KEY),
  ]);
  return post;
}
```

조회수가 오르면 두 캐시가 동시에 낡는다. 그 글의 단건 캐시(`post:${id}`)도, 인기글 목록(`posts:popular`)의 정렬 순서도 옛것이 된다. 그래서 둘 다 지운다. `Promise.all([, ])` 로 두 삭제를 한꺼번에 보내고 기다린다.

- `{ viewCount: { increment: 1 } }` — Prisma의 원자적 증가다. "지금 값에 1을 더하라"를 DB가 한 번에 처리해, 동시에 여러 요청이 와도 값이 어긋나지 않는다.
- `.catch(() => { throw new NotFoundException(.) })` — 없는 글을 갱신하려 하면 Prisma가 에러를 던진다. 그걸 받아 의미 있는 404로 바꾼다.

이 흐름은 위의 동작 확인 시나리오와 정확히 맞물린다. `GET /posts/1` 을 두 번 부르면 두 번째는 `cache` , 그 뒤 `POST /posts/1/view` 로 조회수를 올리면 캐시가 지워지고, 다시 `GET /posts/1` 을 부르면 `db` 로 돌아와 최신 조회수가 반영된다.

라우트 선언 순서의 함정 — 정적 경로가 먼저

```
@Controller("posts")
export class PostsController {
  constructor(private readonly postsService: PostsService) {}

  @Post()
  create(@Body() dto: CreatePostDto) {
    return this.postsService.create(dto);
  }

  // ⚠ 정적 경로(popular)를 :id 보다 먼저 선언해야 한다 (안 그러면 :id로 잡힘)
  @Get("popular")
  popular() {
    return this.postsService.popular();
  }

  @Get(":id")
  findOne(@Param("id") id: string) {
    return this.postsService.findOne(+id);
  }

  @Post(":id/view")
  view(@Param("id") id: string) {
    return this.postsService.incrementView(+id);
  }
}
```

컨트롤러는 얇다. 요청을 받아 서비스 메서드에 그대로 넘길 뿐이다. 다만 주석에 박힌 경고가 핵심이다.

- `@Get("popular")` 를 `@Get(":id")` 보다 먼저 선언해야 한다. 라우트는 위에서부터 매칭된다. `:id` 가 먼저 있으면 `/posts/popular` 요청이 `id = "popular"` 로 잡혀 엉뚱한 `findOne` 으로 흘러간다. 정적 경로가 동적 경로 (`:id`)보다 위에 와야 한다. 흔히 놓치는 순서 함정이다.
- `@Param("id") id: string` 과 `+id` — 경로 파라미터는 항상 문자열이다. 서비스는 숫자 `id`를 받으므로 `+id` 로 숫자 변환해 넘긴다.

`ValidationPipe` 가 진입점에 전역으로 걸려 있어, `@Post()` 의 `dto` 는 `CreatePostDto` 규칙(`title` 은 비어 있지 않은 문자열, `content` 는 문자열)을 통과해야만 서비스에 닿는다.

크론 스케줄 — @Cron 배치와 콜아웃

통계 도메인은 캐시를 쓰지 않는다. 대신 정해진 주기마다 스스로 도는 스케줄러가 핵심이다.

```

@Injectable()
export class StatsService {
  private readonly logger = new Logger(StatsService.name);

  constructor(private readonly prisma: PrismaService) {}

  // 데모용으로 1분마다 실행. 실무에선 보통 매일 자정:
  // @Cron(CronExpression.EVERY_DAY_AT_MIDNIGHT) 또는 @Cron("0 0 * * *")
  @Cron(CronExpression.EVERY_MINUTE)
  async snapshot() {
    return this.capture("cron");
  }
}

```

- **@Cron(CronExpression.EVERY_MINUTE)** — 이 데코레이터가 `snapshot()` 을 1분마다 자동 실행하게 건다. 루트 모듈의 `ScheduleModule.forRoot()` 가 있어야 비로소 동작한다. 실무용 주기는 주석에 적힌 대로 `EVERY_DAY_AT_MIDNIGHT` 나 크론식 `"0 0 * * *"` 이다.
- **snapshot()** 은 위임만 한다. 실제 집계는 `this.capture("cron")` 에 맡긴다. 크론은 "언제 도는가"만 담당하고, "무엇을 하는가"는 `capture` 로 빼 둔다.

집계 본체와 조회를 보자.

```

// 게시글 수 / 총 조회수를 집계해 스냅샷으로 저장.
async capture(trigger: "cron" | "manual") {
  const [postCount, agg] = await Promise.all([
    this.prisma.post.count(),
    this.prisma.post.aggregate({ _sum: { viewCount: true } }),
  ]);
  const totalViews = agg._sum.viewCount ?? 0;

  const stat = await this.prisma.dailyStat.create({
    data: { postCount, totalViews },
  });
  this.logger.log(
    `[${trigger}] 스냅샷 저장 - 글 ${postCount}개 / 총 조회수 ${totalViews}`
  );
  return stat;
}

findAll() {
  return this.prisma.dailyStat.findMany({ orderBy: { id: "desc" }, take: 20 });
}

```

- **Promise.all([count, aggregate])** — 글 개수와 조회수 합계를 두 쿼리로 동시에 구한다. `aggregate` 의 `_sum` 은 합계가 없으면(글이 0개) `null` 이라, `?? 0` 으로 0을 채운다.
- **dailyStat.create(.)** — 집계 결과를 `DailyStat` 한 행으로 적재한다. 매번 갱신이 아니라 새 행을 쌓는다. 그래서 시간이 지나면 시점별 스냅샷이 차곡차곡 남는다.

- **trigger** 인자 — 이 한 메서드를 크론도 부르고 수동 엔드포인트도 부른다. "cron" 인지 "manual" 인지를 받아 로그에 [cron] 또는 [manual] 로 남긴다. 콘솔에서 누가 이 스냅샷을 찍었는지 한눈에 구분된다.
- **findAll()** — 최근 스냅샷 20개를 최신순으로 돌려준다. GET /stats 가 이걸 부른다.

Logger 는 NestJS가 제공하는 로거다. console.log 대신 쓰면 클래스 이름(StatsService)과 타임스탬프가 자동으로 붙어 로그가 정돈된다.

한 로직, 두 입구 — 크론과 수동 실행

```
@Controller("stats")
export class StatsController {
  constructor(private readonly statsService: StatsService) {}

  @Get() // 적재된 스냅샷 목록
  findAll() {
    return this.statsService.findAll();
  }

  @Post("run") // 1분을 기다리지 않고 즉시 한 번 집계(데모용)
  run() {
    return this.statsService.capture("manual");
  }
}
```

- **GET /stats** — 쌓인 스냅샷 목록을 본다.
- **POST /stats/run** — 크론이 도는 1분을 기다리지 않고 지금 당장 한 번 집계한다. 앞서 빼 둔 capture 를 "manual" 로 부른다. 크론과 같은 로직이지만 입구만 다르다. 이 엔드포인트 덕에 스케줄 로직을 즉시 검증할 수 있다.

동작 확인 — 캐시가 실제로 먹는지 보기

서버를 띄운 뒤 curl 로 흐름을 따라가 보자.

```
# 글 두 개 생성 → 인기글 캐시가 무효화된다
curl -X POST http://localhost:3000/posts -H "Content-Type: application/json" -d
'{"title":"A","content":"."}'
curl -X POST http://localhost:3000/posts -H "Content-Type: application/json" -d
'{"title":"B","content":"."}'

# 인기글 — 처음엔 source:"db", 30초 안에 다시 부르면 source:"cache"
curl http://localhost:3000/posts/popular      # db
curl http://localhost:3000/posts/popular      # cache

# 단건도 같은 패턴
curl http://localhost:3000/posts/1            # db
curl http://localhost:3000/posts/1            # cache
curl -X POST http://localhost:3000/posts/1/view # 조회수 변경 → 캐시 무효화
curl http://localhost:3000/posts/1            # 다시 db (최신 반영)
```

Redis에 실제로 키가 들어갔는지 직접 들여다볼 수도 있다.

```
redis-cli KEYS '*'          # posts:popular, post:1.. 이 보인다
redis-cli GET posts:popular # 캐시된 JSON 확인
```

스케줄은 1분을 기다리거나, 수동으로 즉시 돌린다.

```
curl -X POST http://localhost:3000/stats/run # 즉시 한 번 집계
curl http://localhost:3000/stats            # 적재된 스냅샷 목록
```

정리 표

동작	메서드	캐시에 하는 일	키
인기글 조회	<code>popular()</code>	get → 없으면 set (TTL 30초)	<code>posts:popular</code>
단건 조회	<code>findOne(id)</code>	get → 없으면 set (TTL 30초)	<code>post:\${id}</code>
글 생성	<code>create()</code>	del (목록 무효화)	<code>posts:popular</code>
조회수 증가	<code>incrementView(id)</code>	del × 2 (단건 + 목록)	<code>post:\${id}</code> , <code>posts:popular</code>

엔드포인트	하는 일
POST /posts	글 생성 + 인기글 캐시 무효화
GET /posts/popular	인기글 상위 5개 (cache-aside)
GET /posts/:id	단건 조회 (cache-aside)
POST /posts/:id/view	조회수 +1 + 캐시 2개 무효화
GET /stats	적재된 스냅샷 최근 20개
POST /stats/run	통계 즉시 한 번 집계

흔한 실수

- **Redis를 안 켜고 서버를 띄운다.** → 캐시 스토어 연결이 실패해 앱이 부팅 단계에서 죽는다. `brew services start redis` 로 먼저 Redis를 올린다. "왜 갑자기 안 뜨지?"의 첫 번째 의심 대상이다.
- **ScheduleModule.forRoot() 를 빼먹는다.** → @Cron 을 붙여도 타이머가 안 걸려 아무 로그도 안 찍힌다.
- **쓰기 후 캐시를 안 지운다.** → DB는 바뀌었는데 캐시에 옛 값이 남아, TTL이 끝날 때까지 stale 데이터가 나간다. `create · incrementView` 의 `del` 이 그래서 있다.
- **@Get("popular") 를 @Get(":id") 아래에 둔다.** → /posts/popular 가 `id = "popular"` 로 잡혀 엉뚱한 핸들러로 간다. 정적 경로를 동적 경로보다 위에 선언한다.
- **TTL 단위를 헛갈린다.** → 이 스토어의 TTL은 밀리초다. 30 이라 쓰면 30밀리초 만에 만료된다. 30초는 `30_000` 이다.

직접 해보기

1. `findOne` 이 캐시 hit일 때 콘솔에 `cache hit: post:3` 같은 로그를 남기게 해 보자. 어떤 요청이 DB를 치고 어떤 요청이 캐시로 끝나는지 터미널에서 바로 보인다.
2. 인기글 TTL만 60초로 늘려 보자. `set` 의 세 번째 인자만 `60_000` 으로 바꾸면 된다. 단건 캐시는 30초 그대로 두고 둘이 다르게 만료되는지 확인한다.
3. 크론 주기를 `CronExpression.EVERY_30_SECONDS` 로 바꿔 30초마다 스냅샷이 쌓이게 한 뒤, `GET /stats` 로 행이 더 빨리 늘어나는지 본다. 확인 후엔 다시 `EVERY_MINUTE` 로 되돌린다.

정리

- 캐시 어사이드는 읽기는 "캐시 먼저, 없으면 DB 채우기", 쓰기는 "DB 바꾼 뒤 캐시 삭제" 두 규칙으로 요약된다.

- 전역 `CacheModule` 에 Redis 스토어를 물리고 `CACHE_MANAGER` 를 주입하면, 서비스 어디서나 `get · set · del` 로 캐시를 다룬다. TTL은 오래된 캐시의 안전망이다.
- `ScheduleModule.forRoot()` 와 `@Cron` 으로 메서드를 주기 실행하고, 핵심 로직을 별도 메서드로 빼 두면 수동 엔드포인트로도 같은 일을 즉시 돌릴 수 있다.

다음 장(**15 · 실전 종합**)에서는 13장의 인증 가드와 이 장의 캐싱을 하나의 게시판 API에 통합하고, Swagger 문서-페이지 네이션까지 더해 NestJS 과정을 마무리한다.